Introduction

Why do we need high-performance computing?

- natural sciences:
  - everlasting pursuit of realism
  - simulations help us better understand nature
  - simulations are much more cost and time effective compared to real-world experiments
  - applications
    - weather forecasting, material sciences,
    - high-energy physics, astrophysics, nuclear physics,
    - chemistry, lattice QCD, biochemistry, life sciences, genomics, medicine
- data analytics
  - large quantities of data are collected
  - retrieving knowledge from data
  - confirmation of models
  - statistical and artificial intelligence modelling
  - applications:
    - artificial intelligence, deep learning
    - image and signal processing, computer vision, robotics

Motivation

- ever growing need for enormous computing and data processing resources
- performance development over time: top500.org
- Europe
  - EuroHPC JU initiatives
  - HPC systems (petascale systems, exascale systems)
  - AI factories (19 + 2 giga)
- Slovenia
  - VEGA (IZUM, Maribor), 10 PFLOPS, 30 mio. EUR
  - new AI factory system (IZUM, Maribor), projected approx. 100 PFLOPS (CPU 10%), 135 mio. EUR
  - goals
    - Upgrade existing HPC capabilities
    - Provide infrastructure for open research data
    - Data storage for Slovenian R&D
    - Offer computational capacities to industry
    - International cooperation
  - Arnes cluster
    - hardware
    - software
      - SLURM batch system
      - AlmaLinux
      - user specific SW in modules
      - light virtualization (Apptainer)
  - HPC applications
    - examples:
      - CERN: CMS, ATLAS
      - OpenFOAM, ANSYS
      - Gromacs, Quantum espresso
      - AI: Keras, TensorFlow, PyTorch
- reasons for HPC
  - faster time to solution (response time)
  - solve bigger computing problems (in the same time)
- motivation for parallel processing
  - effective use of machine resources
  - cost efficiencies
  - overcoming memory constraints
  - HPC = parallel HW + concurrency + performance
- our approach
  - How does my application scale?
  - I have a specific problem and no adequate SW package. How to solve it more efficiently?
  - How to develop such nice tools and packages?
  - There is a new technology available.
    - Can my problem make use of it?
    - How to adapt my program for it?
  - Working on a cluster during the course
    - the course topic directly addresses clusters
    - we all use the same HW (multi-core nodes with GPUs)
    - the system is well-maintained by professional admins (HW, SW installation)
    - you can access the cluster from anywhere
    - you have valid access for the whole school year, you can use the systems for other courses as well (without reservation), for extension a mentor should write to support@sling.si

Pervasive parallelism

- von Neumann architecture: CPU, memory, I/O
- for a long time (until mid-2000s), the performance of computer systems increased through miniaturization (relays, vacuum tubes, increasingly smaller and more numerous transistors, raising the clock frequency) and improvements in hardware.
- Dennard scaling applied
  - when you shrink a transistor its dimension, voltage, and current shrink
  - reducing the size of a transistor by half increases their number fourfold, each transistor operates twice as fast (shorter connections), while the amount of dissipated heat remains unchanged
  - around mid-2000s, shrinking transistors caused leakage currents and heat problems, voltage could no longer scale down, power density started rising
- characteristics of processors over time (graph)
- Moore's law: system performance doubles every 18 months (graph)
- a problem arises with heat dissipation when processor power consumption exceeds 130 W
- multi-core architectures begin to appear around mid-2000s, which are more energy-efficient
- by increasing the number of processor cores, the validity of Moore's law is maintained

Multi-core processors are more energy efficient than single-core ones

- a processor's power consumption depends on the clock frequency (f), supply voltage (U), and the circuit's capacitance (C)
- a higher frequency means more switching events and therefore higher energy consumption
- if pushed too far, chips overheat and begin to operate unreliably

$$P = U_0 I = U_0 \frac{de}{dt} = U_0 C \frac{dU}{dt}$$

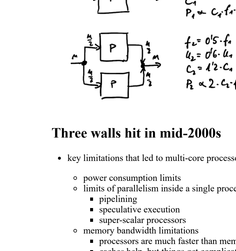
$$U = U_0 \sin(2\pi f t)$$

$$\frac{dU}{dt} = 2\pi f U_0 \cos(2\pi f t)$$

$$P = 2\pi C f U_0^2 \cos(2\pi f t)$$

$$P = kCfU_0^2$$

- two processor cores can operate at a lower voltage, but they have higher capacitance
- with two processor cores, we can perform approximately the same amount of work at half the clock frequency



Three walls hit in mid-2000s

- key limitations that led to multi-core processors:
  - power consumption limits
  - limits of parallelism inside a single processor
    - pipelining
    - speculative execution
    - super-scalar processors
  - memory bandwidth limitations
    - processors are much faster than memory
    - caches help, but things get complicated on multi-core processors
    - scalability problems
- state of the software
  - support for developing parallel software
    - despite many years of research, automatic parallelization does not work well
    - compilers capable of automatically converting our sequential programs into parallel ones are still in development and perform poorly
    - libraries that support parallelization
    - new programming languages with built-in support for parallelism
  - solution
    - to make good use of new architectures, we must write parallel programs
    - new processors include mechanisms that can be efficiently run parallel programs (hardware threads)
    - to achieve good results, we must understand the architecture well
    - how to write code that will remain efficient on future processors

Historical trends

- hardware is naturally parallel
  - parallelism in hardware has been present since the earliest computers
  - great sophistication in mainframe and vector supercomputers (late 1980)
- miniaturization
  - Intel 4004: 4-bit microprocessor with 2,300 transistors
  - Today million times more transistors, a lot of potential
  - Improvements over the years:
    - word sizes,
    - super-scalar capabilities, vector instructions, out-of-order execution, deep pipelines, parallel arithmetic units, hardware multithreading,
    - caches, cache prefetching, virtual memory controllers, page table walking, memory access controllers,- graphics processing units
- term supercomputer is first used during development of Cray-1, 1976, 10 mio. USD, Los Alamos National Laboratory
- at the end of seventies, they are introduced to petrol and automotive industry
- in eighties they come to business world
  - faster computations gives competitive advantage
  - less experiments means cheaper development
  - faster development of new products
- first supercomputers were vector computers
  - high price, slow development
  - last purchased system in Slovenia: CONVEX SP1000/XA-64, 1992
- today supercomputers are distributed computer systems
  - massive production, faster development
  - cheaper systems, step-by-step upgrade
  - become most popular in nineties
  - each core is capable of 10-20 GFLOPS/s
  - VEGA
- modern parallel computers
  - standard computer in institutes: DEC VAX 11/780, 1 MFLOPS
  - 1981: Cosmic Cube: 64 Intel 8086 processors (XT), 5 – 10 MFLOPS, for a half of the DEC VAX price
  - 1986: Connection Machine, Thinking Machine Corporation: 1 CPU, many ALUs
  - 1994: Beowulf, NASA: 16 standard Intel DX processors connected to 10 Mbit Ethernet, Linux
  - 1996: cluster for less than 50k USD is capable of 1 GFLOPS
  - 2024: majority of top 10 supercomputers has large GPU partitions